



AKADEMIA GÓRNICZO-HUTNICZA
IM. STANISŁAWA STASZICA W KRAKOWIE

Lexical analysis – Part 1

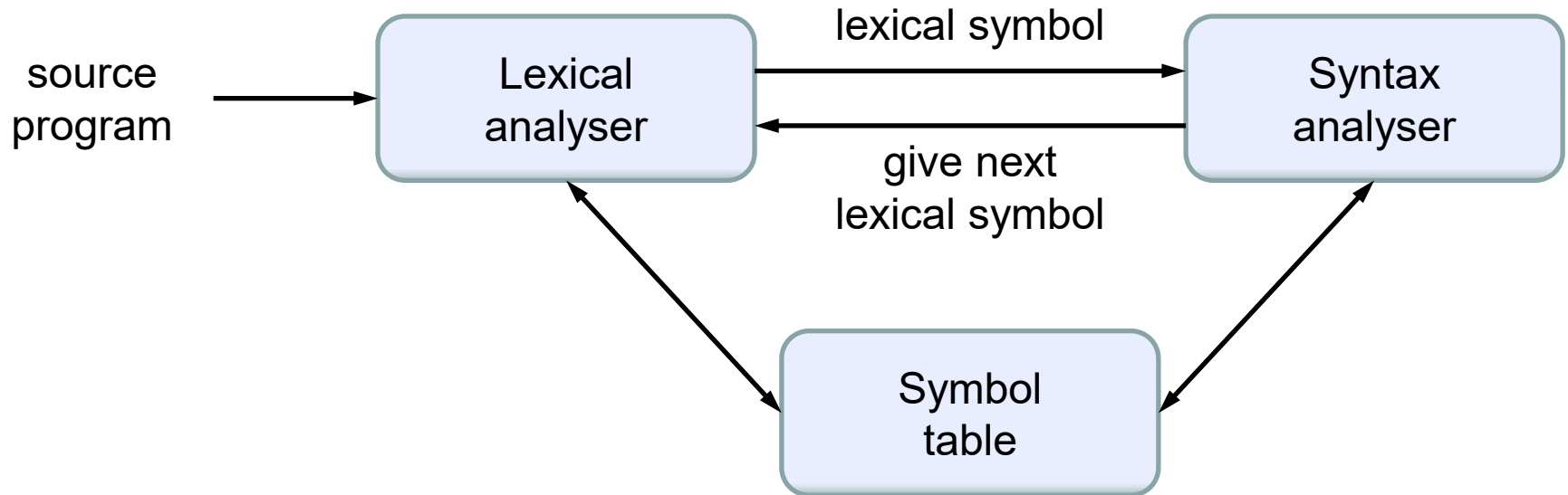
Compilation Theory

Tasks of lexical analyser

Tasks of lexical analyser:

- extraction of lexical symbol (tokens)
- discarding comments
- discarding whitespaces (spaces, tabs, newline characters ...)
- correlating errors reported by a compiler with line numbers
- creating copies of input text (source text) altogether with error messages
- sometimes realisation of preprocessing functionality, macrodefinitions expansion

Cooperation with parser



Separation into lexical analysis and parsing

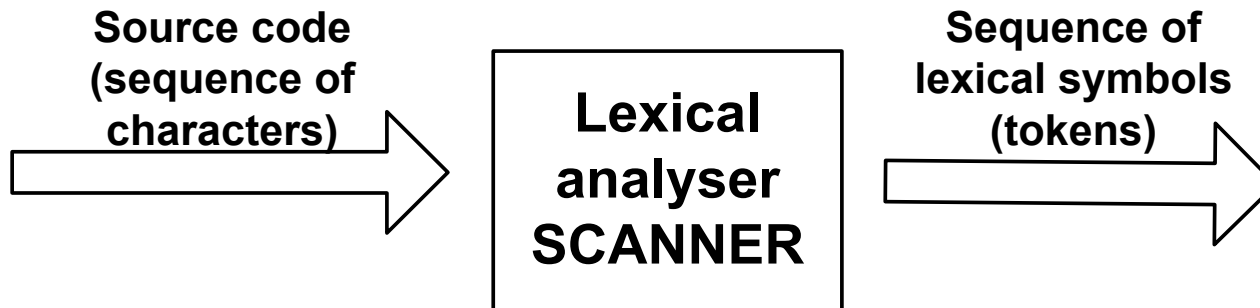
- Simplicity of design
- Improved efficiency
- Enhanced portability

Separation of source program analysis into two separate modules:

- lexical analysis
- syntactic analysis

results in execution of both of them with more effective algorithms, as these algorithms significantly differ and are based on different formal and implementation assumptions.

Tasks of lexical analysis



Example:

Input: cost := (price + tax) * 0.98

Output: $\underline{ID}_1 \underline{:=} (\underline{ID}_2 + ID_3) * \underline{NUM}_4$

Symbol table:

Address	Name/value	Characteristics	Additional information
1	cost	variable	
2	price	variable	
3	tax	variable	
4	0.98	constant	

Tasks of lexical analysis

Example:

Input: cost := (price + tax) * 0.98

Output: $ID_1 \underline{:=} (ID_2 + ID_3) * \underline{NUM}_4$

Scanner input:

=>

Scanner output:

cost

(ID, 1)

:=

(EQU,)

(

(LPAR,)

price

(ID, 2)

+

(PLUS,)

tax

(ID, 3)

)

(RPAR,)

*

(MUL,)

0.98

(NUM, 4)

Basic concepts

- Token
 - `<name,attribute>` pair
 - stream of tokens is passed to parser
 - attribute might be a structure/class of several attributes
- Pattern
 - Describes the strings for the lexemes of the token
- Lexem
 - a sequence of characters that matches the pattern of the token

Basic concepts: token, lexem and pattern

Example:

```
const pi2 = 6.2832;
```

token	lexem	pattern
<u>CONST</u> (keyword)	const	const
<u>ID</u>	pi2	<i><u>letter</u> (<u>letter</u> <u>digit</u>)*</i>
<u>RELOP</u>	=	< > <= >= = <>
<u>NUM</u>	6.2832	<i><u>digit</u>⁺ (. <u>digit</u>⁺)? ((E e) (+ -)? <u>digit</u>⁺)?</i>

```
source:    const pi2 = 6.2832
lexems:    const pi2 = 6.2832
tokens:    CONST ID RELOP NUM
```


Scanner specification

In practice we distinguish three types of tokens

- Literals
- General tokens
- Reserved keywords

Scanner specification

- Literals
 - Lexems are one-character
 - Token can be represented by its one-character lexem

literals = { '+', '-', '*', '/' }

literals = "+-*/"

Scanner specification

- General tokens
 - One token matches many lexems
 - Specified with regular expressions

- Examples

- ID – matches many identifiers

```
@_(r'[a-zA-Z_]\w*')
```

```
def ID(self,t):
```

```
    return t
```

- NUM – matches many numbers

```
@_(r'\d+')
```

```
def NUM(t):
```

```
    return t
```

Scanner specification

- Reserved keywords
 - One token corresponds to exactly one lexem
 - Lexems are longer than one character
 - Their specification matches also specification of an identifier

```
tokens = { "ID", "EQ", "NEQ", "LE", "GE",  
           "BREAK", "CONTINUE", "IF", "ELSE" }
```

```
ID = r'[a-zA-Z_][a-zA-Z0-9_]*'
```

```
ID['break'] = 'BREAK'
```

```
ID['continue'] = 'CONTINUE'
```

```
ID['if'] = 'IF'
```

```
ID['else'] = 'ELSE'
```

Scanner specification

- Pattern to be **avoided** – individual rules for reserved keywords:

BREAK = r'break'

CONTINUE = r'continue'

IF = r'if'

ELSE = r'else'

Grammars describing a language

Context-free grammar G of source language:

$$G = \langle V, \Sigma, P, S \rangle$$

where:

$$\Sigma = \{ A, B, \dots, Z, a, b, \dots, z, 0, 1, \dots, 9, +, -, *, /, .., (,), \dots \}$$

Grammar G is transformed to a family of grammars:

$$G_S = \langle V_S, \Sigma_S, P_S, S_S \rangle \text{ - syntactic grammar}$$

$$\Sigma_S = \{ \Sigma_{S_i} \mid \Sigma_{S_i} \text{ is a token} \} \text{ - tokens are terminals of syntactic grammar}$$

$$G_i = \langle V_i, \Sigma_i, P_i, S_i \rangle \text{ - lexical grammars (one grammar per token)}$$

$$S_i = \Sigma_{S_i}$$

$$\Sigma_{S_i} \in V_i, \Sigma_i \subset \Sigma$$

i -th token Σ_{S_i} is a start symbol (nonterminal) of i -th lexical grammar

Grammars describing a language

Grammar of source language $G = \langle V, \Sigma, P, S \rangle$

(1) Syntactic grammar: $G_S = \langle V_S, \Sigma_S, P_S, S_S \rangle$

(2) Family of lexical grammars $G_i = \langle V_i, \Sigma_i, P_i, S_i \rangle$ one grammar for each token

$$V = V_S \cup \bigcup_{i=1}^n V_i$$

$$\Sigma = \bigcup_{i=1}^n \Sigma_i$$

$$P = P_S \cup \bigcup_{i=1}^n P_i$$

$$S = S_S$$

Grammar of source language, G arises from „substitution” lexical grammars G_i to syntactic grammar G_S .

Difficulties in building lexical analyser

Example:

In certain programming languages keywords are not reserved (FORTRAN, PL/I). Below string is correct in PL/I language:

```
IF THEN THEN THEN = ELSE ; ELSE  
ELSE = THEN;
```


Difficulties in building lexical analyser

Example:

In FORTRAN all spaces (but spaces inside strings) are always ignored. Variable names do not require their declarations. Type of variable is determined on the basis of the first letter of the variable name.

Let us compare:

1) DO 5 I = 1 . 25 is equivalent to DO5I = 1.25
(assignment instruction, DO5I – real variable)

2) DO 5 I = 1, 25 "for,, loop instruction, it is equivalent to:

for I := 1 to 25 do

begin

...

...

label...

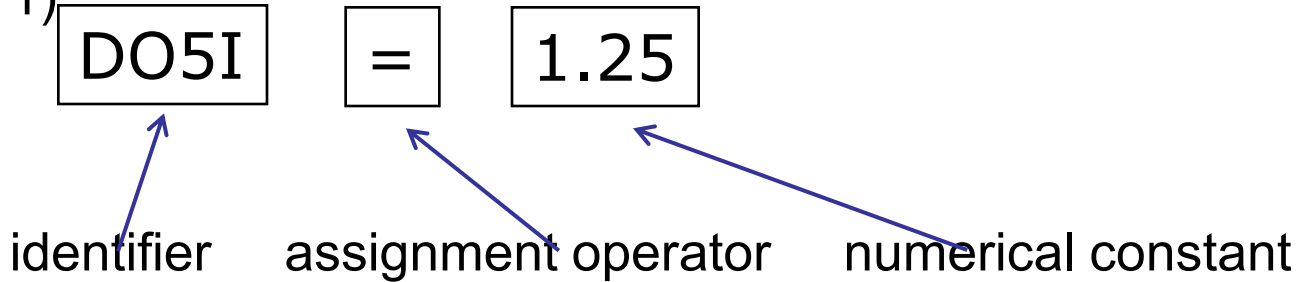


5: end;

Difficulties in building lexical analyser

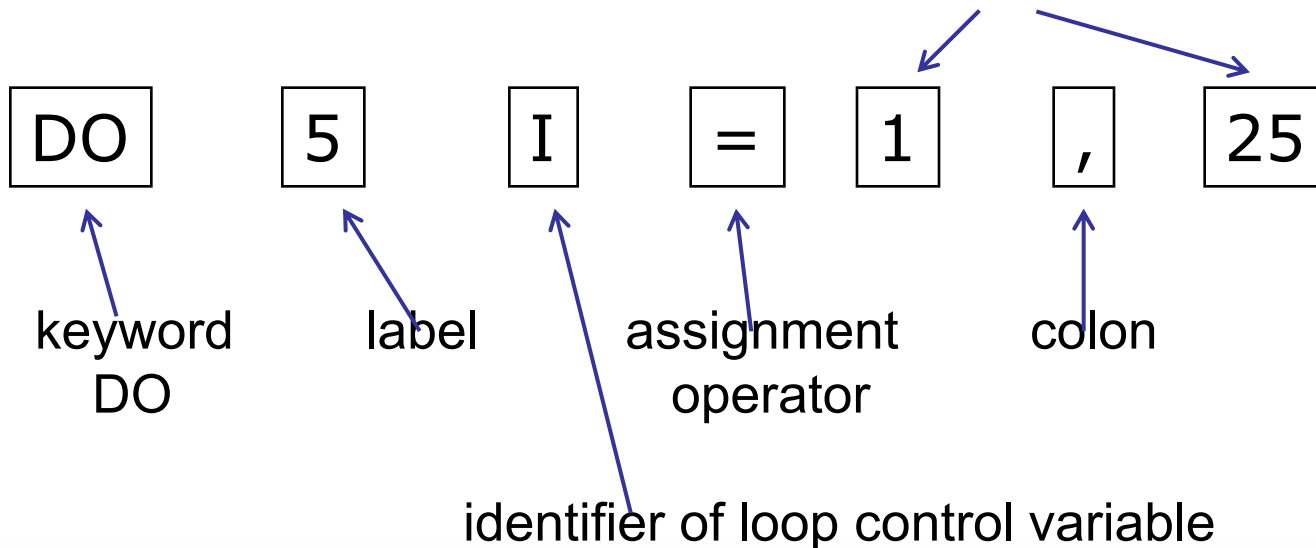
Example, cont.:

1)



2)

expressions (here constants)



Example, cont.:

DO 5 I = 1 { . \n }



After reading characters DO token „keyword DO" cannot be matched until right context is examined and comma is found (then "DO" is matched) or dot or new line character (then we have assignment instruction).



Difficulties in building lexical analyser

Example:

DECLARE(ARG₁, ..., ARG_N)

- Keyword
- Array reference

Difficulties in building lexical analyser

- Python is a semi-fixed format language
 - Its parsing requires two tokens: INDENT and DEDENT
- Tokens INDENT and DEDENT do not correspond to any specific lexem

Difficulties in building lexical analyser

Generation of INDENT and DEDENT tokens

- Push 0 on the top of the stack (it is never popped)
- Read first row
- Numbers pushed onto the stack form an increasing sequence of numbers (from the bottom to the top of the stack)

Difficulties in building lexical analyser

Generation of INDENT and DEDENT tokens

- Compute indentation level of the next logical row and compare with the top of the stack
 - If numbers are equal: do nothing
 - If indentation level is greater than the top of the stack: push it on the stack and generate INDENT token
 - If indentation level is lower than the top of the stack: pop the numbers from the stack, until the indentation level is equal to the number on the top of the stack

Difficulties in building lexical analyser

Generation of INDENT and DEDENT tokens

- When a number is popped from the stack: generate DEDENT token.
- When end of the file is reached: for each positive number remaining on the stack also generate DEDENT token

Difficulties in building lexical analyser

- Complicated rules governing their emission
- Implementation in scanner
 - Needs a separate post-processing phase
- Python's syntax has three INDENT states:
 - NO_INDENT = 0: no colon hence no need to indent
 - MAY_INDENT = 1: "if 1: go()" - simple statements have a COLON but no need for an indent
 - MUST_INDENT = 2: "if 1:\n go()" - complex statements have a COLON NEWLINE and must indent

Difficulties in building lexical analyser

- T TYPE_NAME ID
- T (x) variable declaration function call
- T * x; pointer declaration multiplication
- (T) * x type cast multiplication

Regular definitions

To describe patterns for scanner we use regular definitions:

$$d_1 \rightarrow r_1$$

$$d_2 \rightarrow r_2$$

.....

$$d_n \rightarrow r_n$$

where:

d_i - unique name

r_i - regular expression over alphabet $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$

Regular definitions – example(1)

Unsigned constant in Pascal:

digit $\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

digits $\rightarrow \text{digit digit }^*$

fractional-part $\rightarrow . \text{digits} \mid \varepsilon$

exponent $\rightarrow (E \mid e) (+ \mid - \mid \varepsilon) \text{digits} \mid \varepsilon$

num $\rightarrow \text{digits fractional-part exponent}$

Extended set of operators

For ease of notation new operators in regular expressions are introduced, e.g.:

w^+ - denotes one or more occurrences of pattern w

$$w^+ = w w^*$$

$w?$ - denotes zero or one occurrence of pattern w

$$w? = w \mid \varepsilon$$

Regular definitions – example (2)

Specification of unsigned constant in Pascal using extended operator set with regular expressions:

digit $\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

digits $\rightarrow \text{digit}^+$

fractional-part $\rightarrow (. \text{digits}) ?$

exponent $\rightarrow ((E \mid e) (+ \mid -) ? \text{digits}) ?$

num $\rightarrow \text{digits} \text{fractional-part} \text{exponent}$

Recognizing tokens

Syntactic grammar:

$stmt \rightarrow \underline{if} \ expr \ \underline{then} \ stmt \mid \underline{if} \ expr \ \underline{then} \ stmt \ \underline{else} \ stmt \mid \varepsilon$

$expr \rightarrow term \ \underline{relop} \ term \mid term$

$term \rightarrow \underline{id} \mid \underline{num}$

Regular definitions:

$\underline{delim} \rightarrow \square \mid \backslash t \mid \backslash n$

$\underline{ws} \rightarrow \underline{delim}^+$

$\underline{letter} \rightarrow a \mid b \mid \dots \mid z \mid A \mid B \mid \dots \mid Z$

$\underline{digit} \rightarrow 0 \mid 1 \mid \dots \mid 9$

$\underline{if} \rightarrow if$

$\underline{then} \rightarrow then$

$\underline{else} \rightarrow else$

$\underline{relop} \rightarrow < \mid <= \mid <> \mid > \mid >= \mid =$

$\underline{id} \rightarrow \underline{letter} \ (\underline{letter} \mid \underline{digit})^*$

$\underline{num} \rightarrow \underline{digit}^+ \ (\cdot \underline{digit}^+) ? \ (E \ (+ \mid -) ? \underline{digit}^+) ?$

Regular definitions in practice

- Easy to obtain in flex and Jflex
- Rarely used in PLY
 - constructs provided by **re** library sufficient in most cases
 - possible to achieve using decorators, somewhat „complicated” feature

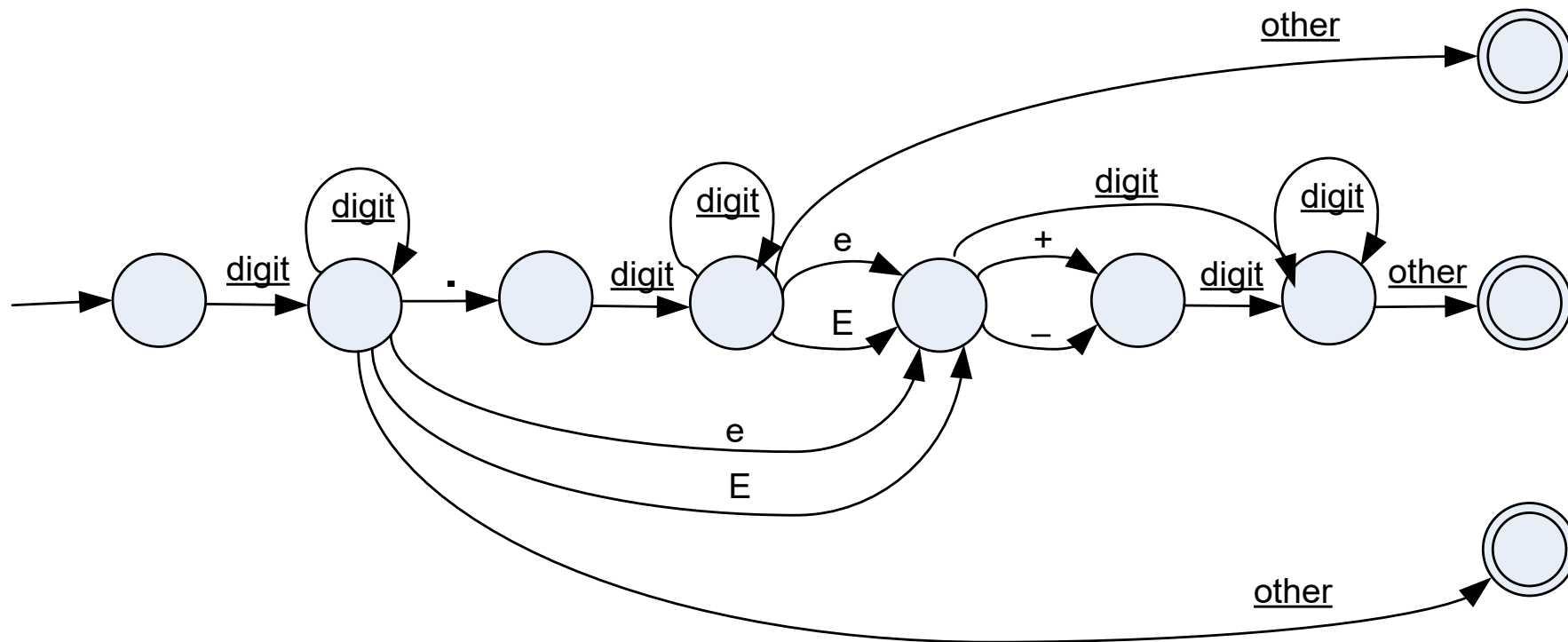
```
from ply.lex import TOKEN
digit = r'([0-9])'
nondigit = r'[_A-Za-z]'
identifier = r'(' + nondigit + r'(' + digit + r'|' + nondigit + r')*)'
@TOKEN(identifier)
def t_ID(t):
```

...

Transition diagrams (1)

Unsigned constant in Pascal

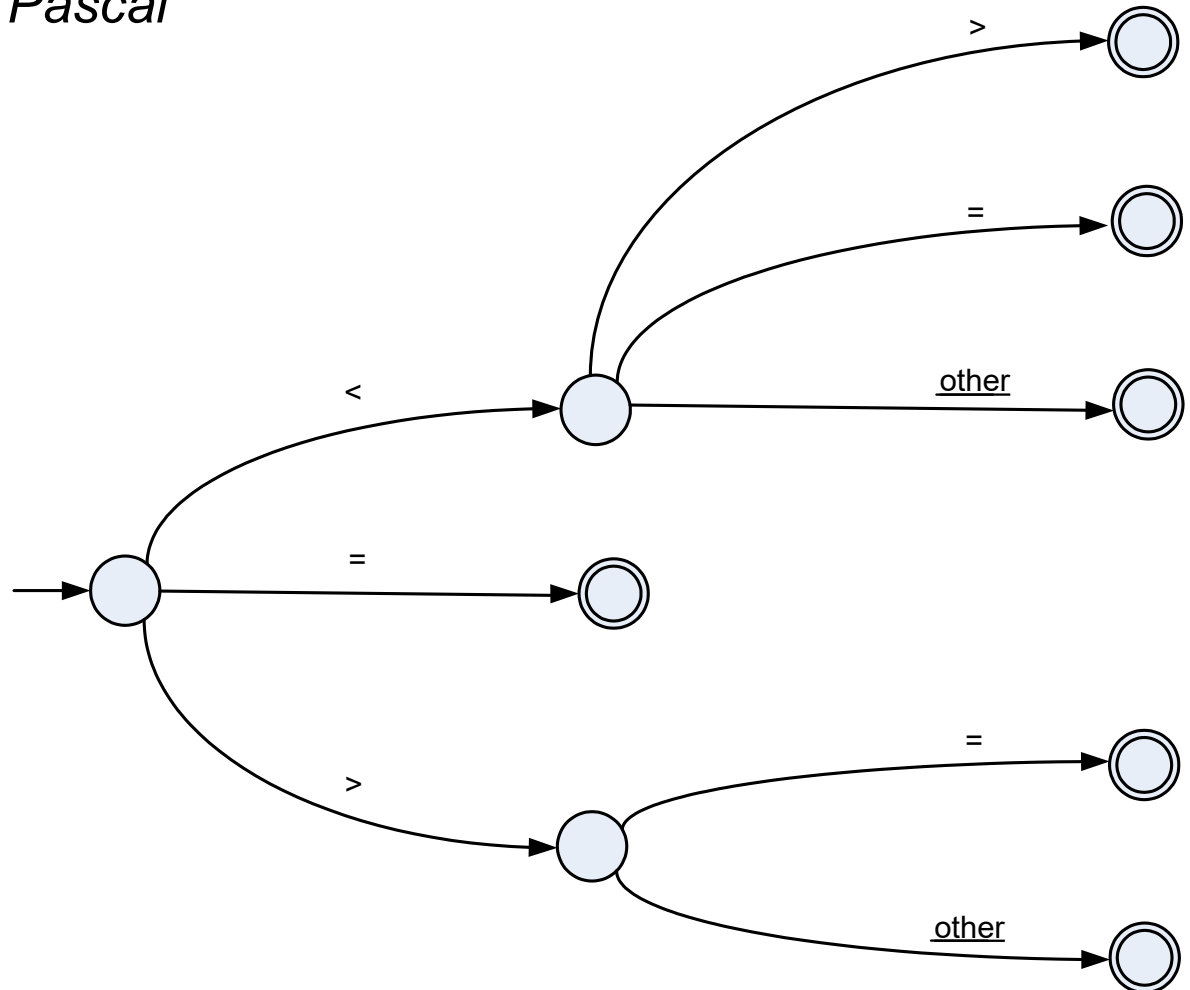
digit⁺ (. digit⁺)? ((e|E) (+|-)? digit⁺)?



Transition diagrams (2)

Relational operators in Pascal

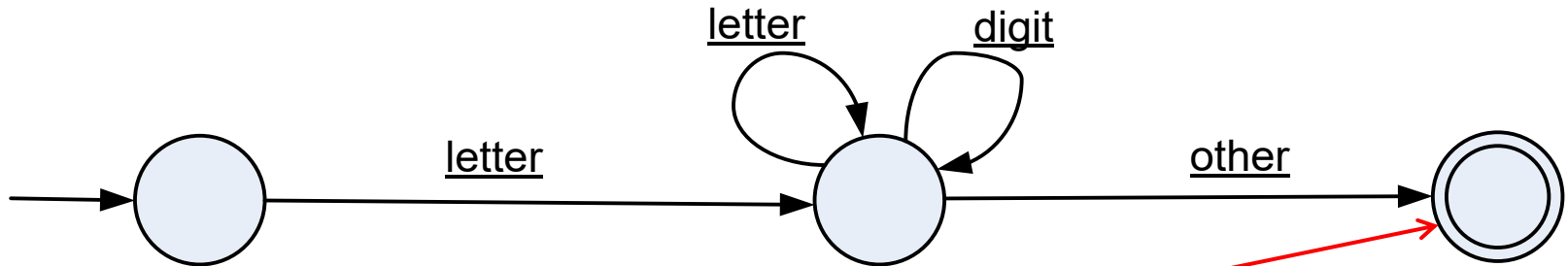
< | <= | <> | = | >= | >



Transition diagrams (3)

Identifier

letter (letter | digit)*

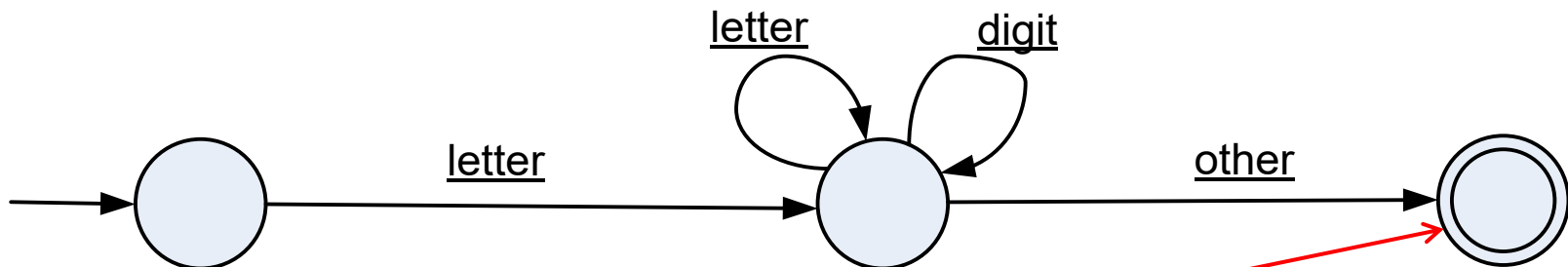


unread last read symbol to input;
check, whether lexem is a keyword;
if yes – return corresponding keyword;
if no – check, whether identifier is already in
symbol table;
if present – return address of its position;
if not present – create new entry
and enter identifier to symbol table.

Transition diagrams (4)

Identifier

letter (letter | digit)*



unread last read symbol to input;
check, whether lexem is a keyword ...

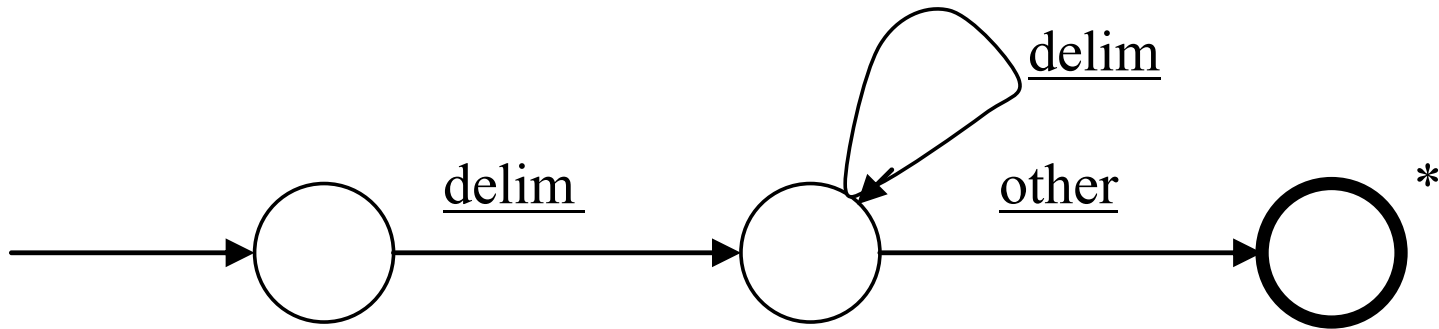
There is a technique other than putting keywords in a table and checking each identifier whether it is not a keyword.

We can specify for keywords separate patterns and build recognizing automata.

However, the number of states of such a lexical analyser grows rapidly.

Transition diagrams

Discarding white characters



unput the last read character to input;
no other action of lexical analyser is taken